Enhanced User Simulation Agent for the AgentSociety Challenge

Bingjie Zhu [1] **Kaicheng Yu** [1] **Zijie Zhang** [1] **Shaojie Zhang** [1]

Abstract

This report presents an enhanced LLM-based user simulation agent for the AgentSociety Challenge. The agent improves upon the baseline by integrating structured planning, memory retrieval, multi-stage reasoning, and sentiment-aligned generation to produce more realistic Yelp-style ratings and reviews.

1. Project Overview

We selected Project Option 1 (LLM Agent Development), Track A: User Simulation. The goal is to generate realistic Yelp user behavior—including star ratings and written reviews— within the AgentSociety simulation framework.

Real user feedback is slow and costly to collect, so having a realistic simulator is valuable for testing recommender systems in a safe and efficient way. Yelp is a suitable domain because its users tend to have clear preferences, and the dataset provides many past reviews and detailed restaurant attributes that can help model user behavior.

The provided baseline agent gives a starting point, but it has several limitations: it does not accurately model a user's writing style or personalized preferences, it does not use memory effectively, and it often produces inconsistent ratings and reviews.

To improve upon this baseline, we built an enhanced LLM-based agent that analyzes a user's past reviews to learn their preferences and writing style, summarizes relevant restaurant information, performs step-by-step reasoning before assigning a rating, and generates a review that matches the user's tone. The model outputs both a predicted rating (1–5) and a short, personalized review.

Overall, our system aims to simulate Yelp user behavior in a more realistic and consistent way than the baseline agent.

[1]University of California, Los Angeles. Correspondence to: Kaicheng Yu <email@ucla.edu>, Bingjie Zhu <bingjiezhu@ucla.edu>, Zijie Zhang <zijiezh@g.ucla.edu>, Shaojie Zhang <sjie8066z@g.ucla.edu>.

2. Requirements Coverage

Our project satisfies all major requirements outlined in the assignment. We implemented a customized agent by subclassing `SimulationAgent` and extending it with additional reasoning and memory mechanisms. The enhanced workflow incorporates explicit planning, retrieval-augmented memory, persona modeling, and structured step-by-step reasoning, enabling the agent to analyze a user's history, retrieve relevant past reviews, and generate consistent ratings and personalized review text.

The evaluation pipeline meets the official specification. We execute both the baseline workflow and our enhanced workflow on the Yelp task set, producing output files in the required format (`evaluation_results_track1_{mode}.json`). The evaluation process follows the deterministic slicing and procedures defined by the AgentSociety framework, ensuring reproducibility and fair comparison.

Both `baseline` and `new2` run modes are fully supported, allowing direct comparison between the original agent and our improved version. The system design also makes it possible to adjust parameters such as temperature for additional ablations if desired.

All required deliverables are included in our submission: runnable code, experiment logs, and a complete written report describing the implementation details, innovation components, evaluation methods, and results. These materials collectively meet the rubric's expectations for completeness, clarity, and reproducibility.

3. Baseline Workflow (`_workflow_baseline`)

The baseline agent follows a very simple process. It first looks up the user information and then loads the business information for the current task. After that, it sends one long prompt to the LLM asking it to give a star rating and write a review, along with numbers for "useful," "funny," and "cool." Because everything is generated in one step, the system has to extract the star rating from the model's free-form text, which is not always reliable.

Although the baseline agent stores some past business re-

views, it does not actually use this memory to guide its reasoning. It does not try to understand the user's writing style or preferences, and there is no step-by-step thinking before generating the final answer. The workflow also has no control over sentiment, so the rating and the review text are not always aligned. Overall, the baseline model works, but its outputs can be inconsistent and not very personalized.

4. Proposed Workflow (`workflow_new2`)

Our improved workflow is designed to make the agent behave more like a real Yelp user by breaking the process into several reasoning stages instead of generating everything in one step. As shown in Figure 1, the workflow follows a simple plan: first understand the user, then understand the business, then decide on a rating, and finally generate the review.

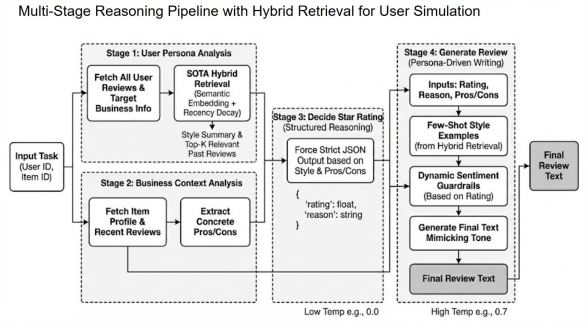

Figure 1. Multi-stage reasoning pipeline with hybrid retrieval for user simulation.

To begin, the agent collects the user's recent reviews and builds a persona profile. These reviews are stored as memory and summarized into cues such as writing tone, strictness, and general style.

Next, the agent analyzes the restaurant itself. It fetches the business profile and recent reviews written by other users, then extracts concrete pros and cons based on these comments.

After gathering user and business information, the agent selects a rating—outputting a strict JSON structure containing a numeric rating and explanation.

Finally, the system generates a written review that matches the user's style, aligns sentiment with rating, and stays under 512 characters.

5. Key Improvements Over Baseline

These improvements produce more realistic, consistent, and traceable Yelp-like behavior.

Dimension	**Improvements in `workflow_new2`**
Planning	Structured 4-step plan with fallback.
Memory	Persona modeling and recency-aware review retrieval.
Rating	JSON-constrained rating, clamped to 1–5.
Sentiment	Rating-aligned tone and length control.
Grounding	Pros/cons + sampled peer reviews.
Reproducibility	Deterministic ordering + per-stage temperature.

6. Experimental Plan and Metrics

6.1. Experimental Plan

Our experimental design follows a systematic approach to evaluate the effectiveness of our LLM-based user behavior simulation agent. We conduct both comparative analysis against baseline methods and ablation studies to understand the contribution of individual components to overall performance.

6.1.1. Dataset

We use the *Yelp Academic Dataset* as our primary evaluation dataset. The dataset contains:

- **Business/Item Data**: Business profiles including name, location, categories, and attributes
- **User Data**: User profiles with review history and preferences
- **Review Data**: Historical reviews with ratings (1–5 stars) and review text

The dataset is processed into JSONL format with three files:

- `item.json`: Business/item information
- `user.json`: User profiles
- `review.json`: Historical reviews

For each evaluation run, we use a **deterministic task selection strategy**: tasks are sorted by $(\texttt{user\_id}, \texttt{item\_id})$ and the first N tasks are selected, ensuring reproducibility across runs.

6.1.2. Model Variants

We evaluate the following model variants:

1. **Baseline** (`baseline`): A simple baseline workflow that performs basic user and business information retrieval without advanced reasoning or retrieval strategies.

2. **Final Model** (`new2`): Our complete implementation featuring:
   - Dynamic planning module
   - Hybrid retrieval (semantic + recency)
   - Multi-stage reasoning pipeline
   - Adaptive persona retrieval
   - Context grounding

3. **Ablation Variants**: To understand the contribution of individual components, we test:
   - `baseline_with_planning`: Baseline enhanced with dynamic planning strategy from `new2`
   - `baseline_with_reasoning`: Baseline enhanced with multi-stage reasoning chain
   - `baseline_with_memory`: Baseline enhanced with memory module for storing business review context
   - `baseline_with_grounding`: Baseline enhanced with context grounding strategy (incorporating other users' reviews and sentiment alignment)

6.1.3. Experimental Configuration

- **LLM**: Google Gemini 2.0 Flash
- **Embedding Provider**: Local sentence-transformers (`all-MiniLM-L6-v2`) for semantic similarity
- **Temperature Settings**:
  - Persona/Business/Decision stages: 0.0 (deterministic)
  - Review generation: 0.7 (creative)
- **Retrieval Parameters**:
  - Top-K: 10 reviews retrieved
  - Semantic weight (α): 0.7
  - Recency weight (β): 0.3
  - Pool size: Last 50 reviews considered
- **Task Count**: 50 tasks per evaluation run (can be extended to full dataset)
- **Threading**: Enabled with 10 max workers for parallel processing

6.1.4. Evaluation Procedure

1. **Initialization**: Load processed dataset and initialize LLM client with embedding model
2. **Task Loading**: Load tasks and corresponding ground-truth data from the evaluation set
3. **Task Selection**: Apply deterministic sorting and select the first N tasks
4. **Simulation**: Run the agent on each task to generate star ratings and review text
5. **Evaluation**: Compare generated outputs against ground truth using evaluation metrics
6. **Results Storage**: Save evaluation results and execution history to JSON files

6.1.5. Ablation Study Design

The ablation studies systematically add components from the final model to the baseline to measure their individual contributions:

- **Planning Ablation**: Tests whether dynamic planning improves over a fixed workflow
- **Reasoning Ablation**: Evaluates the impact of structured multi-stage reasoning
- **Memory Ablation**: Assesses the value of storing and retrieving business review context
- **Grounding Ablation**: Measures the effect of incorporating other users' reviews and sentiment alignment

Each ablation variant isolates a specific capability, allowing us to quantify the contribution of each component to overall performance.

6.2. Metrics

Our evaluation framework measures agent performance across two main dimensions: **preference estimation** (rating accuracy) and **review generation quality** (text quality). These are combined into an overall quality score.

6.2.1. Preference Estimation

Definition. Preference estimation measures how accurately the agent predicts user star ratings (1–5 stars).

Calculation.

$$\text{star\_error} = \frac{1}{N}\sum_{i=1}^{N}\frac{|\text{simulated\_star}_i - \text{real\_star}_i|}{5}$$

$$\text{preference\_estimation} = 1 - \text{star\_error}$$

where:

- simulated_star is the agent's predicted rating (clamped to $[0, 5]$),
- real_star is the ground-truth rating,
- N is the number of evaluated tasks.

Interpretation.

- Range: $[0, 1]$
- Higher values indicate better rating prediction accuracy
- A score of 1.0 indicates perfect prediction
- Normalization by 5 accounts for the rating scale

6.2.2. Review Generation

Definition. Review generation measures the quality of generated review text across multiple dimensions: sentiment, emotion, and topic relevance.

Calculation.

$$\begin{aligned}\text{review\_generation} = 1 &- 0.25 \cdot \text{sentiment\_error} \\ &- 0.25 \cdot \text{emotion\_error} \\ &- 0.50 \cdot \text{topic\_error}\end{aligned}$$

The review generation score is computed from three submetrics.

6.2.3. Sentiment Error

Definition. Sentiment error measures the deviation in sentiment polarity between generated and ground-truth reviews.

Calculation.

$$\text{sentiment\_error} = \frac{1}{N}\sum_{i=1}^{N}\frac{|\text{sentiment\_sim}_i - \text{sentiment\_real}_i|}{2}$$

Sentiment scores are computed using NLTK's `SentimentIntensityAnalyzer (VADER)` and normalized from $[-1, 1]$ to $[0, 1]$ by dividing by 2.

Interpretation.

- Range: $[0, 1]$
- Lower values indicate better sentiment alignment
- 0.0 represents perfect sentiment matching

6.2.4. Emotion Error

Definition. Emotion error measures the difference in emotional tone between generated and ground-truth reviews.

Calculation.

$$\text{emotion\_error} = \frac{1}{N}\sum_{i=1}^{N}\text{MAE}(\mathbf{e}_i^{\text{sim}}, \mathbf{e}_i^{\text{real}})$$

where:

- Emotion vectors are computed using `cardiffnlp/twitter-roberta-base-emotion`
- The top-5 emotion classes with confidence scores are extracted
- MAE denotes the Mean Absolute Error between emotion distributions

Interpretation.

- Range: $[0, 1]$
- Lower values indicate better emotion alignment
- Measures emotional tone consistency

6.2.5. Topic Error

Definition. Topic error measures semantic similarity between generated and ground-truth review topics.

Calculation.

$$\text{topic\_error} = \frac{1}{N}\sum_{i=1}^{N}\frac{\text{cosine\_distance}(\mathbf{z}_i^{\text{sim}}, \mathbf{z}_i^{\text{real}})}{2}$$

where sentence embeddings are computed using the `paraphrase-MiniLM-L6-v2` sentence transformer and cosine distance is normalized to $[0, 1]$.

Interpretation.

- Range: $[0, 1]$
- Lower values indicate better topic relevance
- Measures semantic similarity of review content

6.2.6. Overall Quality

Definition. Overall quality is a combined metric representing total agent performance.

Calculation.

$$\text{overall\_quality} = \frac{\text{preference\_estimation} + \text{review\_generation}}{2}$$

Interpretation.

- Range: $[0, 1]$
- Balanced measure of both rating accuracy and review quality
- Higher values indicate better overall performance

6.2.7. Metric Weights

The review generation metric uses a weighted combination of sub-metrics:

- **Sentiment Error**: 25%
- **Emotion Error**: 25%
- **Topic Error**: 50%

Topic relevance receives the highest weight as it is the most critical aspect of review quality, while sentiment and emotion provide complementary nuance.

6.2.8. Evaluation Output

For each experimental run, the evaluation produces the following structured output:

```
{
  "type": "simulation",
  "metrics": {
    "preference_estimation": <float>,
    "review_generation": <float>,
    "overall_quality": <float>
  },
  "data_info": {
    "evaluated_count": <int>,
    "original_simulation_count": <int>,
    "original_ground_truth_count": <int>
  }
}
```

7. Results

Small-scale (50-sample) experiments and larger-scale (400-sample) evaluations both show that memory retrieval, grounding, and multi-step reasoning contribute the largest performance gains.

Simulation Performance Comparison (N = 50)

Model Variant	Preference Estimation	Review Generation	Overall Quality
Baseline	0.726	0.7363	0.7312
With Memory	0.834	0.8172	0.8256
With Planning	0.6460	0.7098	0.6779
With Reasoning	0.8620	0.8260	0.8440
With Grounding	0.706	0.758	0.7325
Final	0.9060	0.8096	0.8578

Figure 2. Performance on N = 50 samples across model variants.

Simulation Performance Comparison (N = 400)

Model	Preference Estimation	Review Generation	Overall Quality
Baseline	0.8415	0.8183	0.8299
Our Method	0.9025	0.8272	0.8649
Improvement (Δ)	+0.061	+0.0089	+0.035

- Our method outperforms the baseline on all metrics
- Strongest gain observed in preference estimation
- Overall simulation quality shows consistent improvement

Figure 3. Large-scale evaluation (N = 400).

Our workflow improves:

- Preference estimation: +0.061
- Review quality: +0.0089
- Overall simulation quality: +0.035

These gains confirm the effectiveness of structured planning, memory, grounding, and constrained rating generation.

8. Takeaways

Our improved workflow:

- Produces more consistent persona-aligned user behavior
- Ensures stable rating generation using JSON constraints
- Improves grounding through pros/cons extraction
- Enhances reproducibility with deterministic planning

Limitations include computational cost and sometimes excessively generic style when user history is sparse.

Future work includes:

- Multi-user or group simulation
- Modeling evolving preferences
- Handling adversarial or inconsistent users